[Webinar] How Talkdesk Runs Enterprise-Grade OpenTofu with Spacelift ➡ Register Now

TERRAFORM

Terraform .tfvars files: Variables Management with Examples

Omkar Birade
Updated 14 Feb 2025 · 12 min read

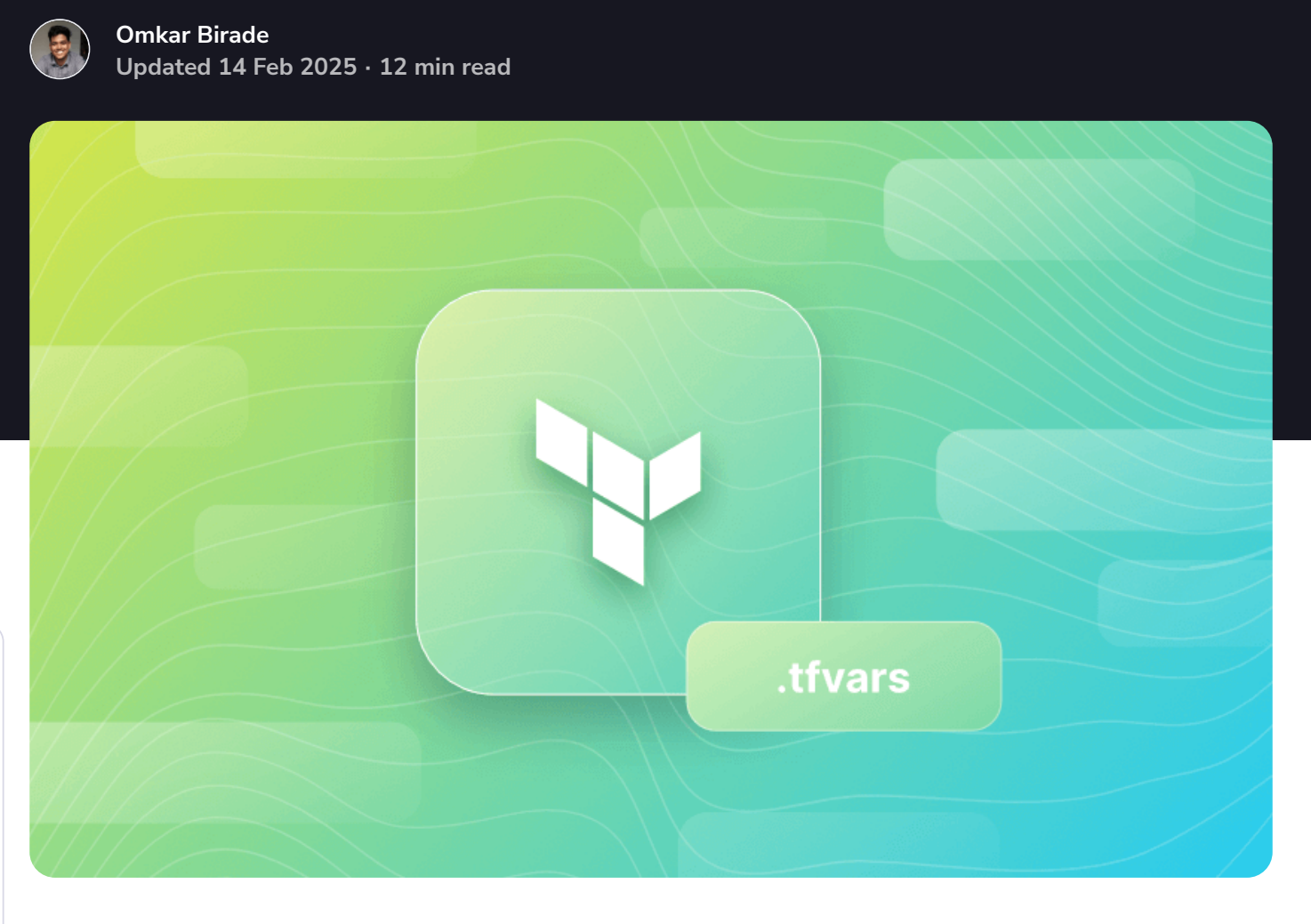


TABLE OF CONTENTS
What is a tfvars file in Terraform?
How to use tfvars files?
How to manage different versions of variable values for different environments
Autoloading tfvars files
Variable loading precedence
Variable.tf files
terraform.tfvars vs variable.tf
How to use tfvars with modules: Child modules and hierarchy of processing
Using Terragrunt to keep Terraform configuration DRY
Why use Spacelift to manage Terraform?
Key points

Subscribe to our Newsletter
Mission Infrastructure newsletter is a monthly digest of the latest posts from our blog, curated to give you the insights you need to advance your infrastructure game.
Your email address
Subscribe

Managing variables effectively is one of the most important aspects of a fantastic IaC project. An IaC project can have multiple modules, all of which would use multiple types of resources, and they could possibly have different values across various environments. To make this setup configurable, the project should be able to use variables and manage them effectively.

Let's dive into understanding everything about the tfvars files and paving the way for creating the most awesome IaC projects.

What is a tfvars file in Terraform?

A `tfvars` **file** in Terraform is used to define input variables for a Terraform configuration. It's a plain text file, typically written in HashiCorp Configuration Language (HCL) or JSON format., that contains variable assignments, making it easy to customize deployments without modifying the core configuration files. Using a tfvars file helps keep your configurations reusable and avoids hardcoding values.

Tfvars files allow us to manage variable assignments systematically in a file with the extension .tfvars or .tfvars.json. Although there are numerous ways to manage variables in Terraform, tfvars files are the best and most common due to their simplicity and effectiveness.

Knowing that the best way to learn new things is to do them ourselves, we'll approach this hands-on to understand the various ways of managing variables in Terraform.

How to use tfvars files?

In the code example below, we will set up a project to spin up an EC2 instance and learn about declaring and managing variables for it. There are code snippets as well as git repository code to refer to at each step.

1. Set up the base code

Let's spin up a simple EC2 instance through Terraform.

```
resource "aws_instance" "billing_server" {
  ami = data.aws_ami.ubuntu.id
  instance_type = "t2.micro"
  tags = {
    "service" = "billing"
  }
}
```

Check out the base code **here**.

2. Introduce variables

The base code works perfectly. However, imagine working in multiple environments, such as test, staging, and production. We would prefer to deploy a larger instance in production and a smaller instance in the test environment to save money. This means that the code remains the same except for the instance type, which varies depending on the environment.

We already know that variables can be used to solve this problem. To keep things running, we'll create a variable for the instance type and set its default value to "t2.micro".

```
variable "instance_type" {
 type = string
 default = "t2.micro"
 description = "EC2 instance type"
}
```

The variable can be referred to by using the var keyword followed by . and then the <variable_name>.

```
resource "aws_instance" "billing_server" {
 ami = data.aws_ami.ubuntu.id
 instance_type = var.instance_type
 tags = {
        "service" = "billing"
    }
}
```

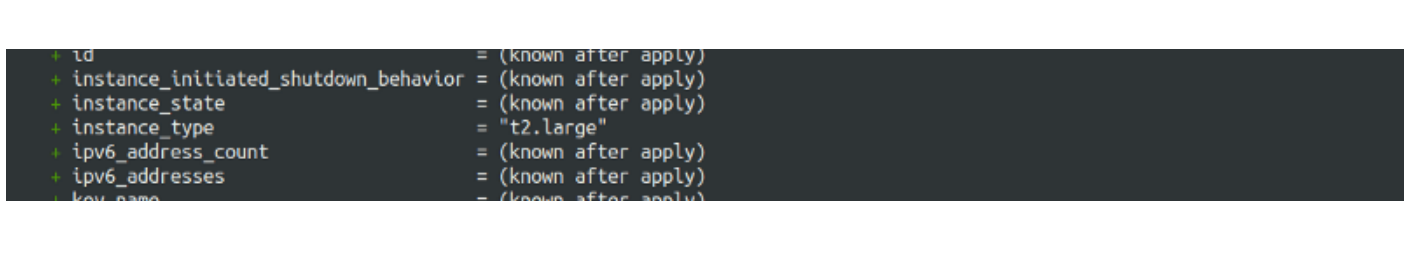
Introducing-variable-github-code

The above code works but still relies on the default value. What if we want to change the instance type just for the production environment? How do we pass it a different value when deploying it to production?

There are numerous ways to accomplish this. Let's look at them one by one.

- Assigning a value through the command line.

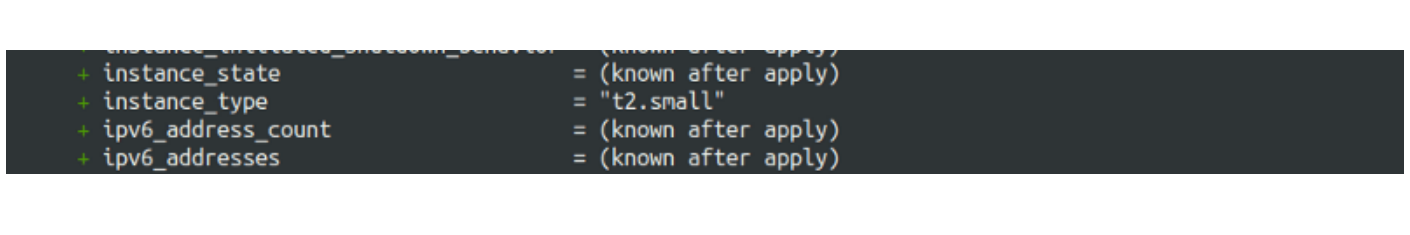
```
terraform plan -var="instance_type=t2.large"
```



```
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_state                       = (known after apply)
  + instance_type                        = "t2.large"
  + ipv6_address_count                   = (known after apply)
  + ipv6_addresses                       = (known after apply)
```

- Using environment variables to set a value.

```
TF_VAR_instance_type="t2.small" terraform plan
```



```
  + instance_state                       = (known after apply)
  + instance_type                        = "t2.small"
  + ipv6_address_count                   = (known after apply)
  + ipv6_addresses                       = (known after apply)
```

Sure, both of these methods work. But *imagine typing in values for **10 to 20 variables** for **5–10 different environments**!* 🤯

Even if we save the command somewhere, consider how time-consuming simple changes like changing a few values would be. This is where *tfvars* files come in handy. Terraform can load variable definitions from these files automatically. Let's have a look.

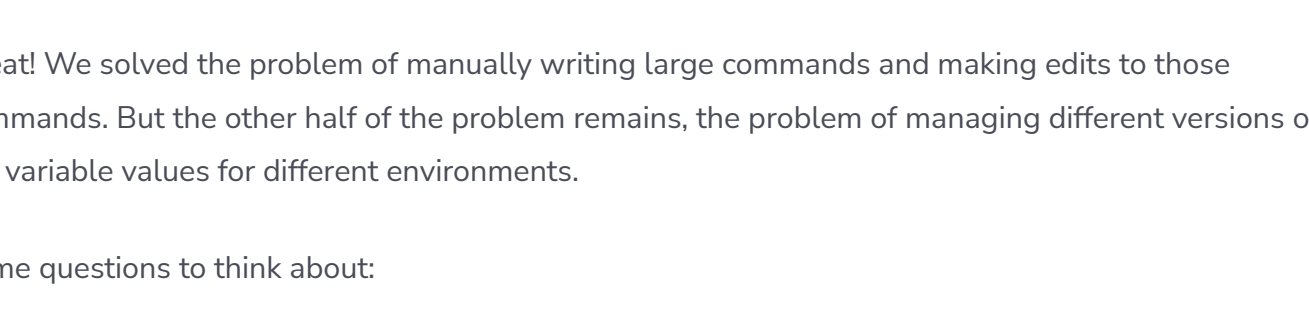
Note: It's not uncommon to have 10-20 variables in your IaC when deploying to multiple environments.

> 💡 **Pro tip**
>
> Whenever the number of variables increases significantly, it is wise to break down the project into various small projects.

3. Create tfvars files

Let's assign the variable a value using the terraform.tfvars file.

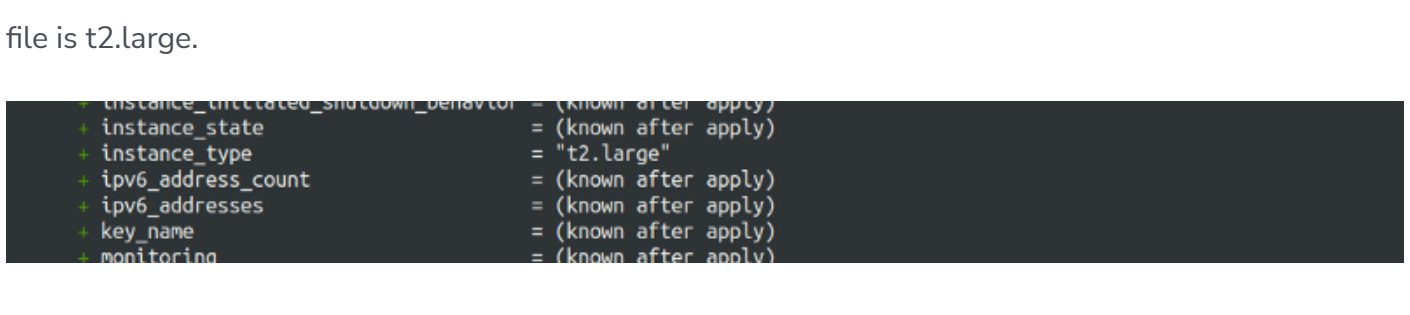
We will start by creating a file named terraform.tfvars and assigning the instance_type variable a value in this file.



```
instance_type = "t2.large"
```

Add-terraform-tfvars-file-github-code

Let's see if Terraform picks the assigned value in terraform.tfvars file when we run `terraform plan`. Remember the default instance_type variable value is t2.micro and value assigned in terraform.tfvars file is t2.large.



```
  + instance_state                       = (known after apply)
  + instance_type                        = "t2.large"
  + ipv6_address_count                   = (known after apply)
  + ipv6_addresses                       = (known after apply)
  + key_name                             = (known after apply)
```

Notice that not only did Terraform load the value from the terraform.tfvars file, but it also overwrote the default value assigned to the variable.

How to manage different versions of variable values for different environments

Great! We solved the problem of manually writing large commands and making edits to those commands. But the other half of the problem remains, the problem of managing different versions of the variable values for different environments.

Some questions to think about:

- Should we manually edit the tfvars file every time we deploy to a different environment?
- Would this solution scale for 10-20 variables with frequent deployments to various environments?

Continuously updating the tfvars file may become tiresome.

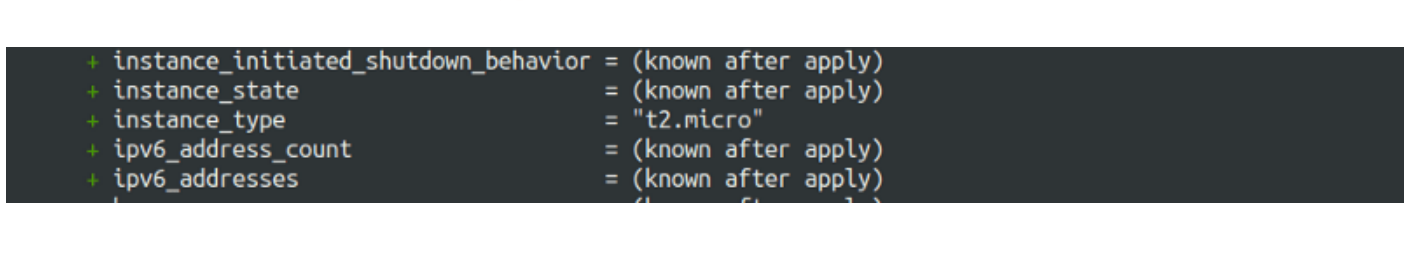
What if we created multiple files like test.tfvars, staging.tfvars, production.tfvars, and then passed these variables into Terraform at runtime.

Let's create two versions of the tfvars file by the name dev.tfvars and prod.tfvars

We will set the instance_type as t2.large for dev version and t2.xlarge for production.

Add-dev-prod-versions-github-code

Let's try running `terraform plan` again and see which version terraform picks up by default. Take a guess before you run this command 😉



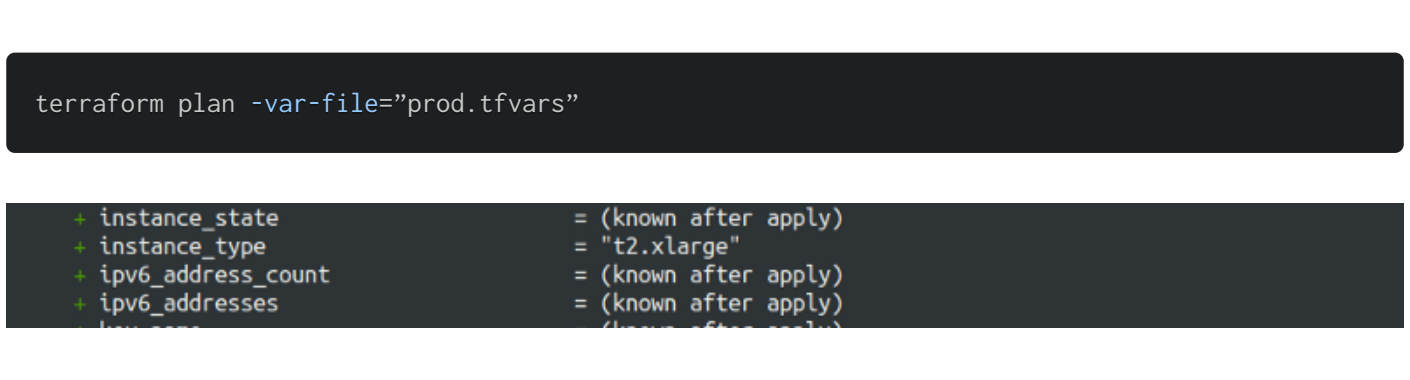
```
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_state                       = (known after apply)
  + instance_type                        = "t2.micro"
  + ipv6_address_count                   = (known after apply)
  + ipv6_addresses                       = (known after apply)
```

It picked neither of the versions we defined; it actually went with the t2.micro default value that is assigned to the variable.

How do we ask Terraform to use either of our versions then?

This can be done using the *-var-file* flag to specify the tfvars file to load. For instance, to load the production version, we will use the following command.

```
terraform plan -var-file=”prod.tfvars”
```



```
  + instance_state                       = (known after apply)
  + instance_type                        = "t2.xlarge"
  + ipv6_address_count                   = (known after apply)
  + ipv6_addresses                       = (known after apply)
```

Awesome! We learned how to pass values to Terraform variables in a variety of ways and solved our many to many problems of managing multiple variables for many environments.

Autoloading tfvars files

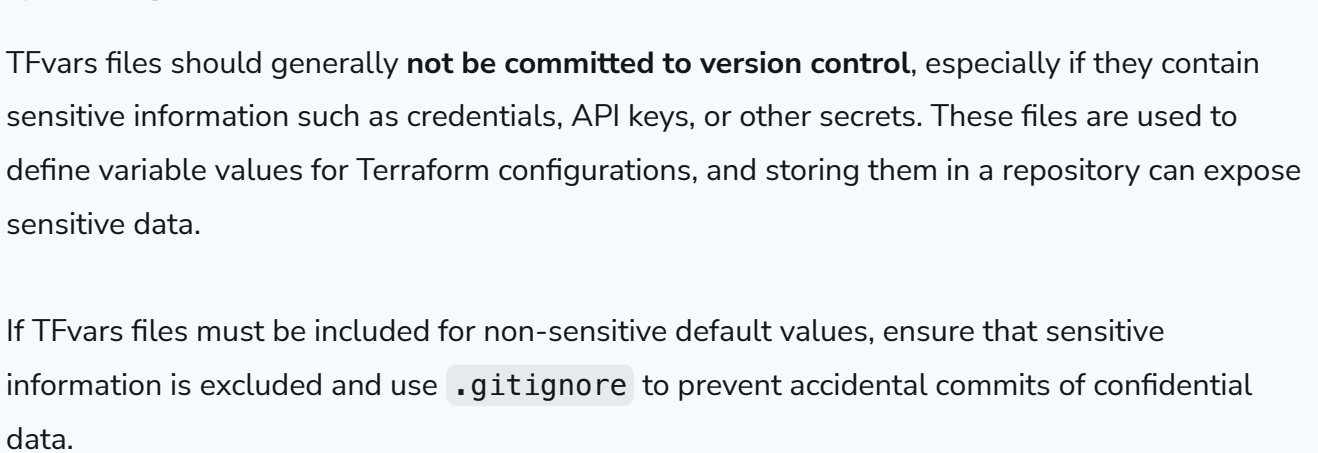
Terraform uses .tfvars files to manage variable assignments, which must be manually loaded with the -var-file flag unless named terraform.tfvars. An *.auto.tfvars* file is automatically loaded by Terraform, making it useful for defining default or environment-specific variables without requiring explicit inclusion in commands.

Terraform auto loads *tfvars* files only when they are named terraform.tfvars or terraform.tfvars.json. Can we ensure that *tfvars* files following custom naming schemes are loaded automatically?

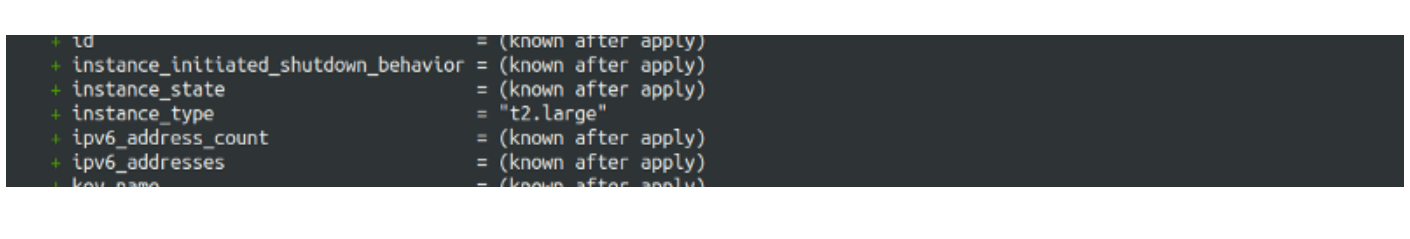
It is possible to name your file whatever you wish and have Terraform load it automatically. All we have to do is provide the file name with .auto.tfvars as an extension. Let's try it out.

We will rename the dev.tfvars file to dev.auto.tfvars and run `terraform plan` on it. The expectation is that Terraform should automatically pick the value t2.large from the dev.auto.tfvars file instead of the default value t2.micro.

Rename-dev-to-dev-auto-tfvars-github-code



```
instance_type = "t2.large"
```



```
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_state                       = (known after apply)
  + instance_type                        = "t2.large"
  + ipv6_address_count                   = (known after apply)
  + ipv6_addresses                       = (known after apply)
```

Notice that Terraform automatically loads out our dev.auto.tfvars file as expected.

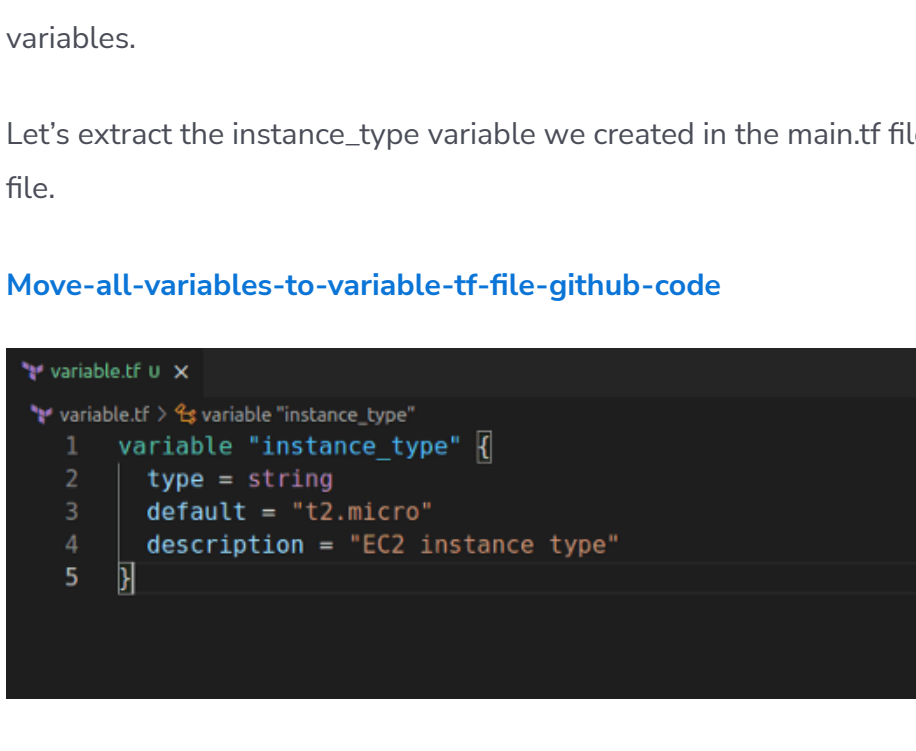
Terraform can load variable definitions from these files automatically if:

- Files named exactly terraform.tfvars or terraform.tfvars.json.
- Any files with names ending in .auto.tfvars or .auto.tfvars.json.

> 💡 **Pro tip: Should TFvars be committed?**
>
> TFvars files should generally **not be committed to version control**, especially if they contain sensitive information such as credentials, API keys, or other secrets. These files are used to define variable values for Terraform configurations, and storing them in a repository can expose sensitive data.
>
> If TFvars files must be included for non-sensitive default values, ensure that sensitive information is excluded and use `.gitignore` to prevent accidental commits of confidential data.

Variable loading precedence

An interesting thing to ponder is the precedence of the various methods of providing variable values.


Any -var and -var-file options on the command line, in the order they are provided.
Any *.auto.tfvars or *.auto.tfvars.json files, processed in lexical order of their filenames
terraform.tfvars.json
terraform.tfvars
Environment variables
Precedence

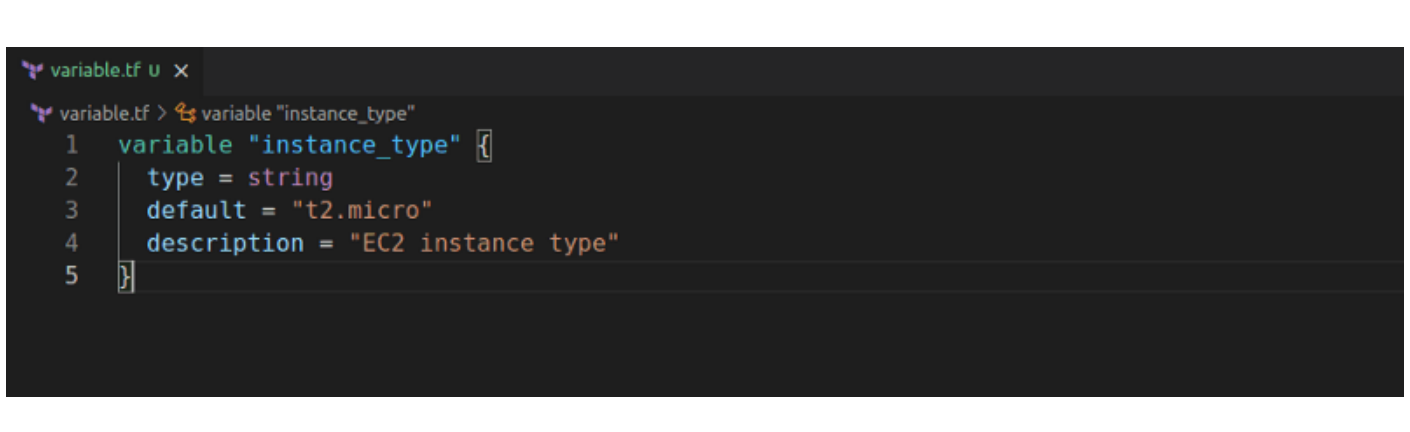
Variable.tf files

A typical Terraform project can have 10-20 variables. With this in mind, it becomes a priority to better manage the structure by logically separating all variables in a single place.

Terraform recommends this single place be the variables.tf file, so we can easily declare and find all variables.

Let's extract the instance_type variable we created in the main.tf file into a new file named variable.tf file.

Move-all-variables-to-variable-tf-file-github-code



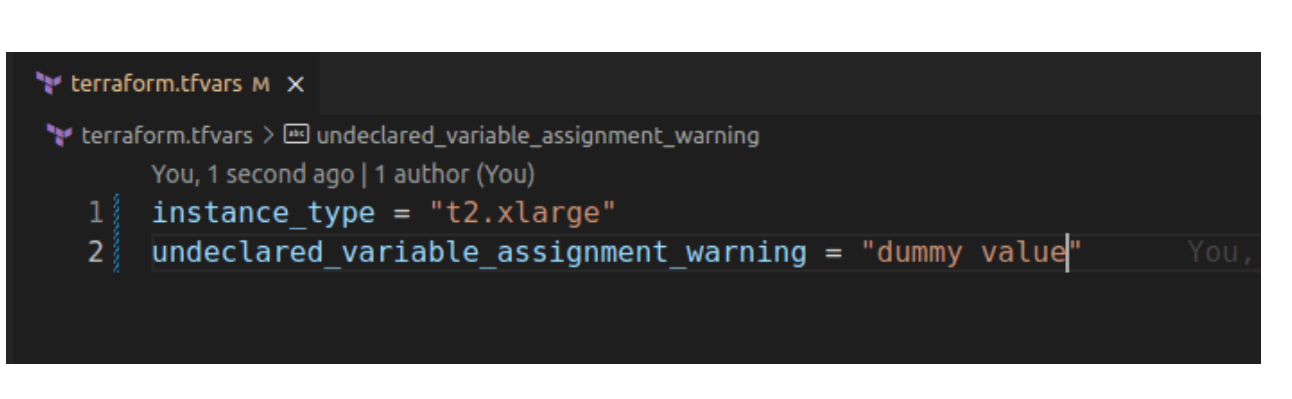
```
variable "instance_type" {
  type = string
  default = "t2.micro"
  description = "EC2 instance type"
}
```

Remember that variable.tf is like any other Terraform file; there is nothing special about it, and it is only used to logically separate variable declarations into a separate file.

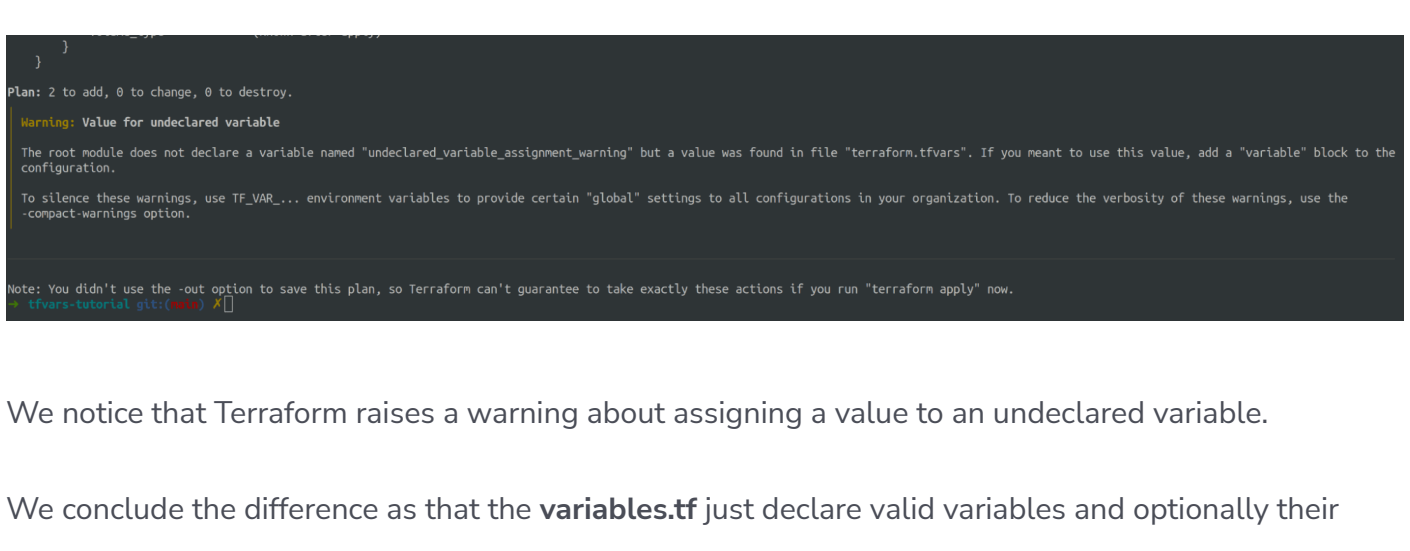
terraform.tfvars vs variable.tf

Do not confuse *terraform.tfvars* files with *variable.tf* files. Both are completely different and serve different purposes.

variable.tf are files where all variables are declared; these might or might not have a default value, while variable.tfvars are files where the variables are provided/assigned a value.

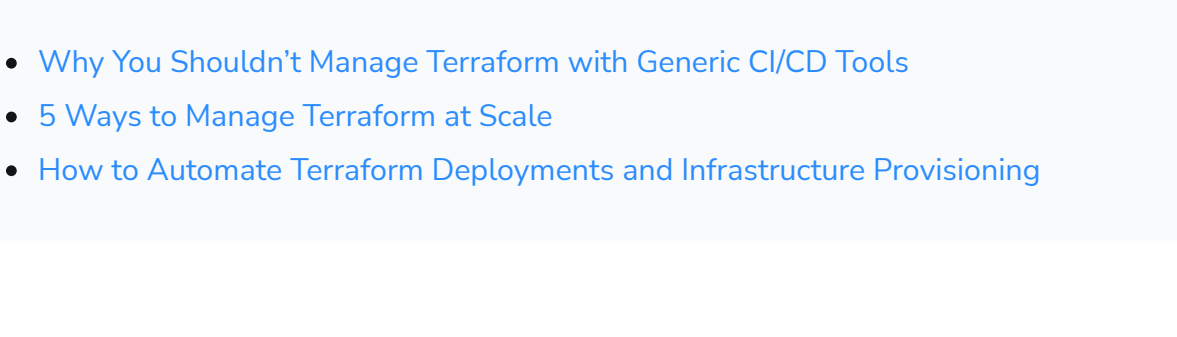


```
variable "instance_type" {
  type = string
  default = "t2.micro"
  description = "EC2 instance type"
}
```

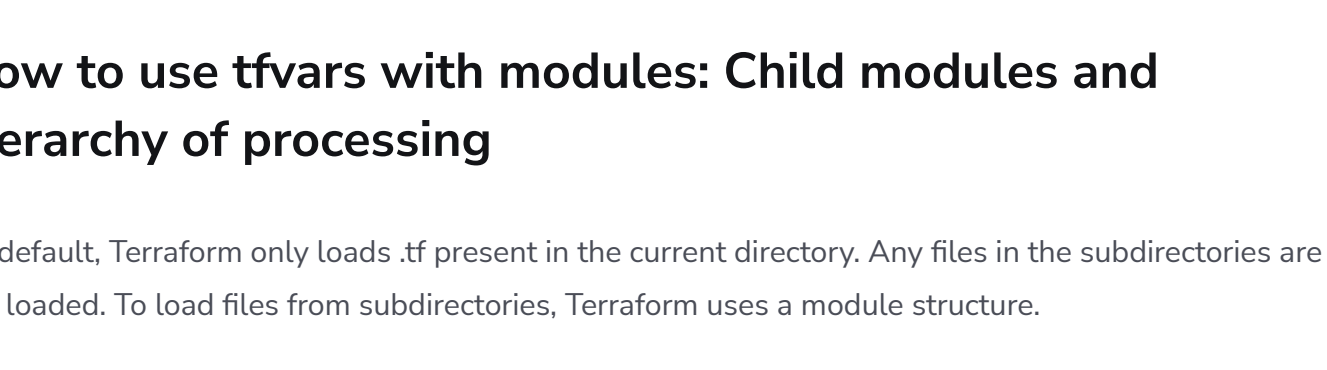


```
instance_type = "t2.large"
```

The difference becomes clear if we try assigning the variable a value that is not declared in the variable.tf file.


terraform.tfvars M ×
terraform.tfvars > undeclared_variable_assignment_warning
You, 1 second ago | 1 author (You)

```
instance_type = "t2.xlarge"
undeclared_variable_assignment_warning = "dummy value"
```



```
Plan: 2 to add, 0 to change, 0 to destroy.

Warning: Value for undeclared variable

The root module does not declare a variable named "undeclared_variable_assignment_warning" but a value was found in file "terraform.tfvars". If you meant to use this value, add a "variable" block to the configuration.

To silence these warnings, use TF_VAR_... environment variables to provide certain "global" settings to all configurations in your organization. To reduce the verbosity of these warnings, use the -compact-warnings option.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
```

We notice that Terraform raises a warning about assigning a value to an undeclared variable.

We conclude the difference as that the **variables.tf** just declare valid variables and optionally their types, and the *tfvars* file assigns them values.

> 💡 **You might also like:**
>
> - Why You Shouldn't Manage Terraform with Generic CI/CD Tools
> - 5 Ways to Manage Terraform at Scale
> - How to Automate Terraform Deployments and Infrastructure Provisioning

How to use tfvars with modules: Child modules and hierarchy of processing

By default, Terraform only loads .tf present in the current directory. Any files in the subdirectories are not loaded. To load files from subdirectories, Terraform uses a module structure.

It's vital to keep in mind that .tfvars values only apply to variables in the root module.

The root .tfvars files are essential and authoritative, while child .tfvars files have no effect.

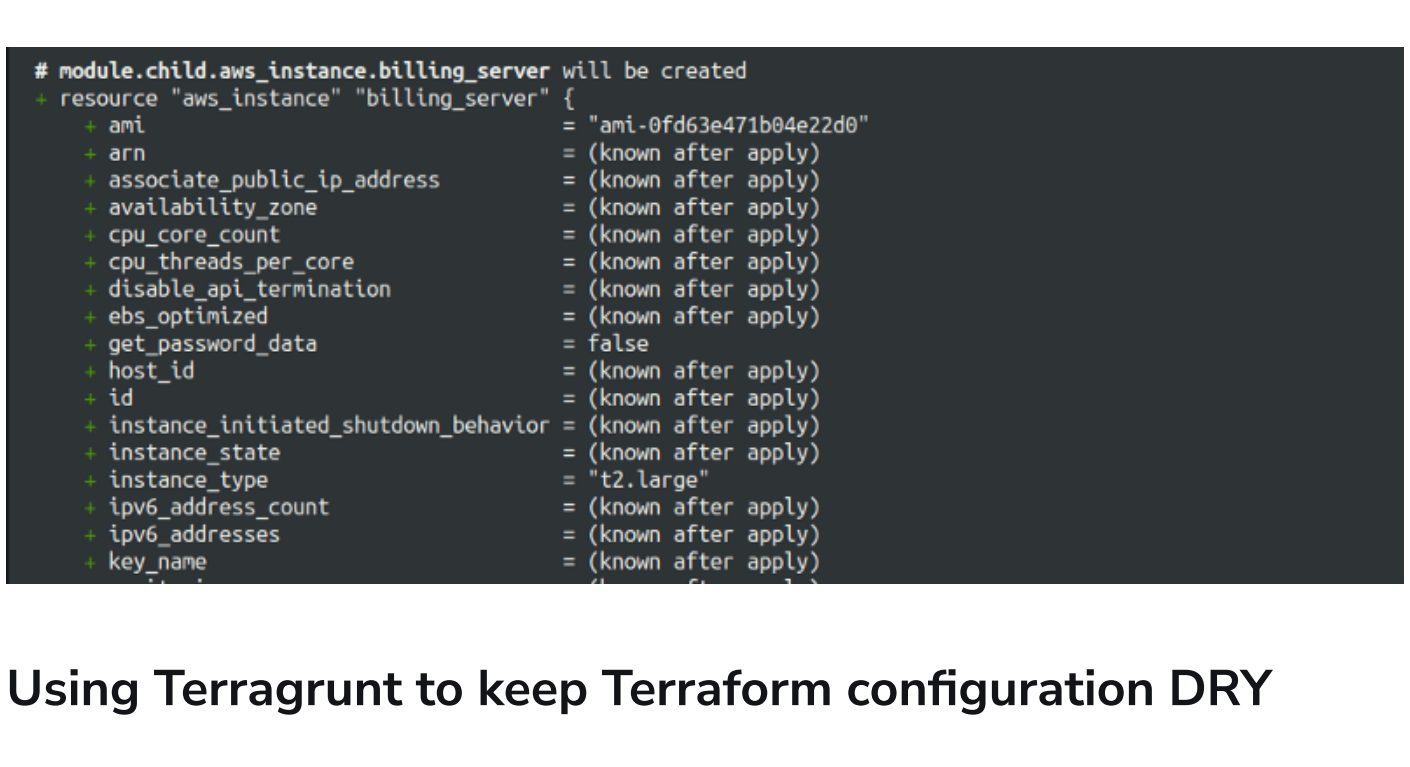
Let's see a code example to understand this better.

Create a child module that spins another EC2 instance.

```
/*file path : ./modules/main.tf*/
resource "aws_instance" "billing_server" {
 ami = data.aws_ami.ubuntu.id
 instance_type = var.instance_type
 tags = {
   "service" = "billing"
 }
}
```

Create another variable.tf and terraform.tfvarsfiles inside the modules directory. Assign the instance_type variable a value as t2.small.

Add-child-module-to-show-hierarchy-github-code



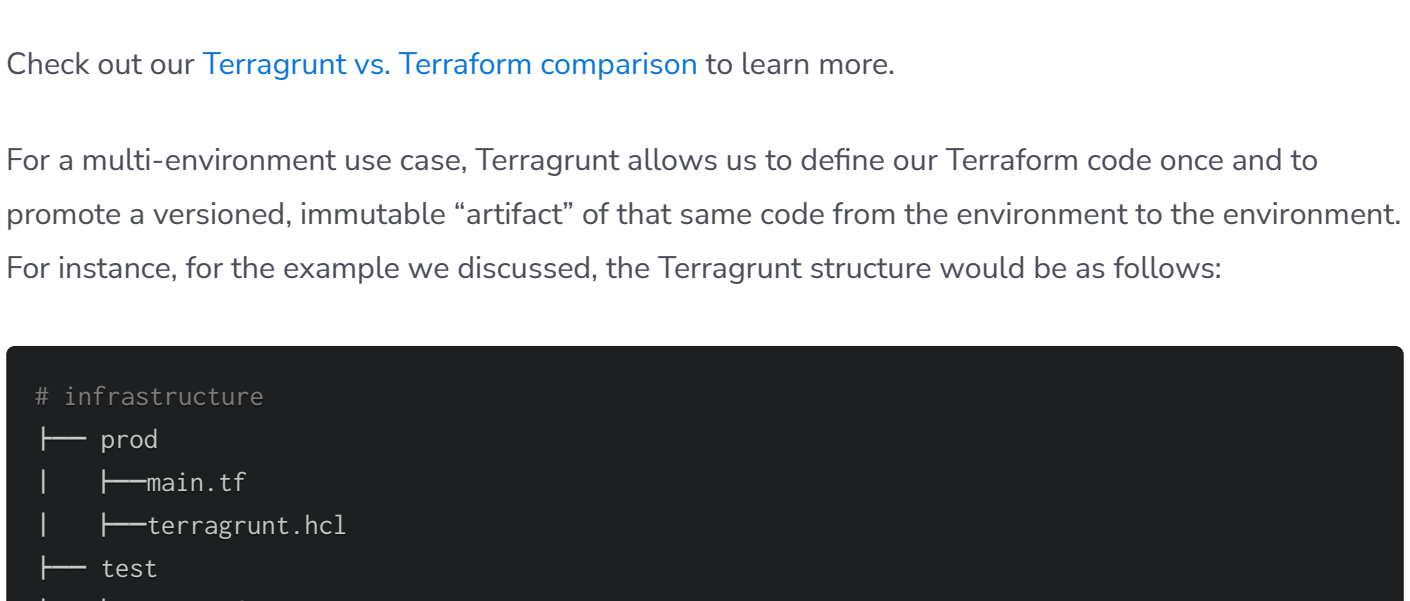
```
instance_type = "t2.small"
```

Call the child module in the root module.

Call-child-module-in-root-module-github-code

Run `terraform init` for Terraform to load the module statically, then run `terraform plan`.

We can notice that the child module *tfvars* file is ignored and the instance type is set as t2.xlarge (root terraform.tfvars) instead of t2.small (child module terraform.tfvars).



```
# module.child.aws_instance.billing_server will be created
+ resource "aws_instance" "billing_server" {
    + ami                                  = "ami-0fd63e471b04e22d0"
    + arn                                  = (known after apply)
    + associate_public_ip_address          = (known after apply)
    + availability_zone                    = (known after apply)
    + cpu_core_count                       = (known after apply)
    + cpu_threads_per_core                 = (known after apply)
    + disable_api_termination              = (known after apply)
    + ebs_optimized                        = (known after apply)
    + get_password_data                    = false
    + host_id                              = (known after apply)
    + id                                   = (known after apply)
    + instance_initiated_shutdown_behavior = (known after apply)
    + instance_state                       = (known after apply)
    + instance_type                        = "t2.large"
    + ipv6_address_count                   = (known after apply)
    + ipv6_addresses                       = (known after apply)
    + key_name                             = (known after apply)
```

Using Terragrunt to keep Terraform configuration DRY

Terragrunt is a tool outside Terraform that allows us to keep the Terraform configuration DRY and reduce variable usage redundancies.

Terragrunt aims to minimize repetition, whether it's in remote backends, multiple environments, or multiple variables per environment. It employs a terragrunt.hcl file to centrally define a common template configuration and generate code from the defined template.

Check out our Terragrunt vs. Terraform comparison to learn more.

For a multi-environment use case, Terragrunt allows us to define our Terraform code once and to promote a versioned, immutable "artifact" of that same code from the environment to the environment. For instance, for the example we discussed, the Terragrunt structure would be as follows:

```
# infrastructure
├── prod
|    ├──main.tf
|    ├──terragrunt.hcl
├── test
|    ├──main.tf
|    ├──terragrunt.hcl
├── qa
|    ├──main.tf
|    ├──terragrunt.hcl
├── dev
|    ├──main.tf
      ├──terragrunt.hcl
```

Inputs

You can set values for your module's input parameters by specifying an inputs block in terragrunt.hcl

```
inputs = {
    instance_type  = "t2.micro"
    instance_count = 10
    tags = {
        Name = "example-app"
    }
}
```

Whenever you run a Terragrunt command, Terragrunt will set any inputs you pass in as environment variables.

```
$ terragrunt apply
/* It is equivalent to:
TF_VAR_instance_type="t2.micro" \
TF_VAR_instance_count=10 \
TF_VAR_tags='{"Name":"example-app"}' \
terraform apply
*/
```

Using inputs, the Terragrunt file prod looks like this:

```
# infrastructure/prod/ec2/terragrunt.hcl
terraform {
  source =
    "github.com:learn-tfvars/infrastructure-modules.git//ec2?ref=v0.0.1"
}
inputs = {
instance_type = "t2.xlarge"
}
```

The terragrunt.hcl file in the test environment will look similar, but it will configure smaller instances type to save money:

```
# infrastructure/test/ec2/terragrunt.hcl
terraform {
  source =
    "github.com:learn-tfvars/infrastructure-modules.git//ec2?ref=v0.0.1"
}
inputs = {
instance_type = "t2.small"
}
```

When we run `terragrunt apply`, Terragrunt downloads the ec2 module into a temporary folder and passes the module with the input variables followed by the `terraform apply` command in that folder. This way, each module in each environment is defined by a single terragrunt.hcl file that solely specifies the Terraform module to deploy and the input variables specific to that environment.

Terragrunt is natively supported on Spacelift. By running your Terragrunt commands on Spacelift, you can extend the capabilities of Terragrunt with the powers of Spacelift. This will provide you access to other powerful features such as Spacelift Contexts and Policies. For example, using Spacelift Contexts, you can define a collection of variables and files and then attach these to Spacelift stack(s). Learn more here.

Why use Spacelift to manage Terraform?

Terraform is really powerful, but to achieve an end-to-end secure Gitops approach, you need to use a product that can run your Terraform workflows. Spacelift takes managing Terraform to the next level by giving you access to a powerful CI/CD workflow and unlocking features such as:

- **Policies (based on Open Policy Agent)** – You can control how many approvals you need for runs, what kind of resources you can create, and what kind of parameters these resources can have, and you can also control the behavior when a pull request is open or merged.
- **Multi-IaC workflows** – Combine Terraform with Kubernetes, Ansible, and other infrastructure-as-code (IaC) tools such as OpenTofu, Pulumi, and CloudFormation, create dependencies among them, and share outputs
- **Build self-service infrastructure** – You can use Blueprints to build self-service infrastructure; simply complete a form to provision infrastructure based on Terraform and other supported tools.
- **Integrations with any third-party tools** – You can integrate with your favorite third-party tools and even build policies for them. For example, see how to Integrate security tools in your workflows using Custom Inputs.

Spacelift enables you to create private workers inside your infrastructure, which helps you execute Spacelift-related workflows on your end. For more information on configuring private workers, refer to the documentation.

You can check it for free by creating a trial account or booking a demo with one of our engineers.

Key points

Terraform provides a number of methods for declaring and assigning variables. Variable.tf and terraform.tfvars files are excellent for externalizing configurations and passing values so they can be easily deployed across multiple environments.

If you need more help with Terraform, check How to Automate Terraform Deployments blog post.

Note: New versions of Terraform are placed under the BUSL license, but everything created before version 1.5.x stays open-source. OpenTofu is an open-source version of Terraform that expands on Terraform's existing concepts and offerings. It is a viable alternative to HashiCorp's Terraform, being forked from Terraform version 1.5.6

Automate Terraform Deployments with Spacelift

Automate your infrastructure provisioning, and build more complex workflows based on Terraform using policy as code, programmatic configuration, context sharing, drift detection, resource visualization, and many more.

Learn more

Written by

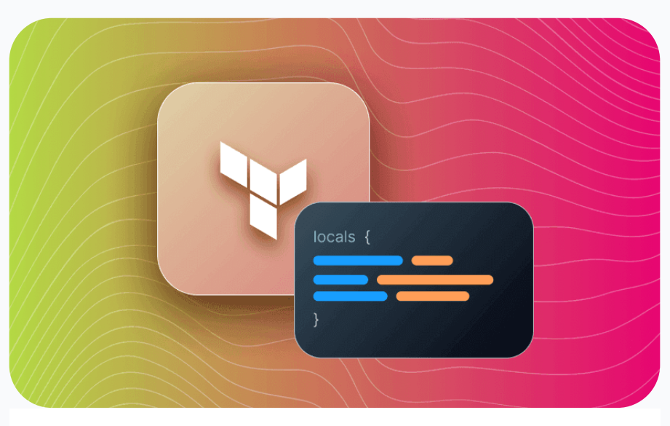
Omkar Birade

Omkar is a CoFounder at Interleap and a seasoned DevSecOps engineer with a passion for DevOps and security. He specializes in AWS DevOps, Terraform, CDKTF and Docker and has held roles ranging from a Fullstack engineer to a DevSecOps Engineer. In his spare time, he takes pleasure in learning new things and sharing his learnings on his personal blog at omkarbirade.medium.com.

Read also

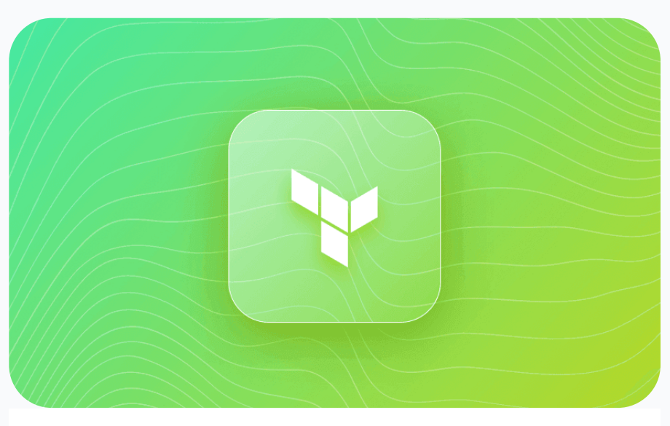
TERRAFORM 14 min read

Terraform Output Values : Complete Guide & Examples



TERRAFORM 8 min read

Terraform Locals: What Are They, How to Use Them



TERRAFORM 24 min read

How to Use Terraform Variables: Examples & Best Practices

Get our newsletter
Subscribe

Product
Documentation
Why Spacelift
How it works
Spacelift Tutorial
Pricing
Customer Case Studies
Integrations
Security
System Status
Product Updates
Test Pilot Program

Company
About Us
Careers
Contact Sales
Partners
Media resources
Accessibility

Learn
Blog
Atlantis Alternative
Terraform Cloud Alternative
Terraform Enterprise Alternative
Spacelift for AWS
Terraform Automation
Achieve Terraform at Scale
CI/CD for Infrastructure
Drift Detection
Newsletter

© 2025 Spacelift, Inc. All rights reserved